Lec-19: Containers, Iterators, and Algorithms

The problem: every data structure is different

You have written arrays, linked lists, BSTs... each with its own traversal code:

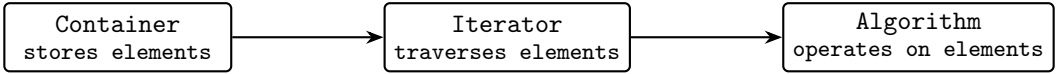
```
// array
for (int i = 0; i < n; i++)
    process(arr[i]);

// linked list
Node* cur = head;
while (cur != nullptr) {
    process(cur->data);
    cur = cur->next;
}
```

Every time you change the container, you rewrite the loop.

What if there were a *uniform* way to traverse **any** container?

The STL idea: separate data from traversal

The C++ Standard Template Library (STL) gives us three building blocks:

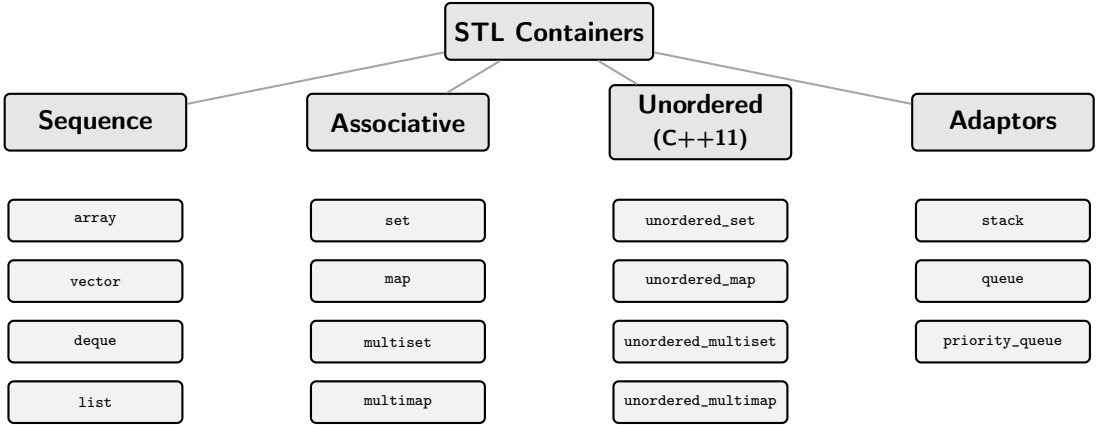


- **Containers** store data (arrays, lists, trees, . . .).
- **Iterators** provide a uniform cursor into any container.
- **Algorithms** (sort, find, . . .) work on iterators — they never need to know which container is behind them.

Why use the standard containers?

- **Productivity** — no need to write (and debug) your own linked list or hash table.
- **Reliability** — implementations are extensively tested; you avoid subtle off-by-one or memory bugs.
- **Performance** — highly optimised by compiler and library vendors; often faster than hand-rolled code.
- **Uniform interface** — `begin()`, `end()`, `size()`, `empty()`, `swap()`, `==` work on *every* container.
- **No external dependencies** — part of the language standard.
- **Readability** — other C++ developers instantly understand `std::vector` or `std::map`.

Container taxonomy



Common container properties

Every STL container provides:

- Default constructor, destructor
- Copy constructor and copy assignment (deep copy)
- Move constructor and move assignment (C++11)
- `begin()` / `end()` returning iterators
- `size()`, `empty()`, `max_size()`
- Content equality (`==`, `!=`)
- Lexicographic comparison (`<`, `<=`, `>`, `>=`)
- `swap()` (constant time for most containers)

Exception: `std::forward_list` has no `size()` method (to keep the node overhead minimal).

Sequence containers overview

Sequence containers store elements in the order you insert them.

- `std::array<T,N>` (C++11) — fixed-size contiguous array (stack allocated)
- `std::vector<T>` — resizable contiguous array (heap allocated)
- `std::deque<T>` — double-ended queue
- `std::list<T>` — doubly-linked list
- `std::forward_list<T>` (C++11) — singly-linked list

Recommendation: Use `std::vector` unless you have a specific reason not to.
Contiguous memory = cache-friendly.

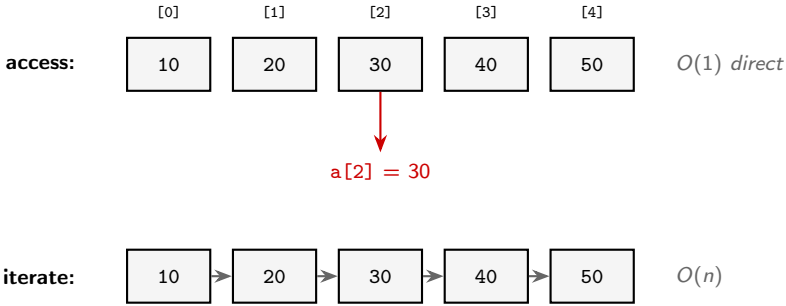
std::array — fixed-size contiguous storage (1/2)

`std::array<T,N>` is a thin wrapper around a C-style array, but with value semantics:

```
#include <array>
#include <algorithm>

std::array<int, 5> a = {7, 3, 1, 9, 4};

a.size(); // 5 (constexpr!)
a[0]; // 7 (no bounds check)
a.at(0); // 7 (throws std::out_of_range)

std::array<int, 5> b = a; // copies all 5 elements
std::sort(a.begin(), a.end()); // a: {1, 3, 4, 7, 9}
a.fill(0); // a: {0, 0, 0, 0, 0}

a.data(); // pointer to underlying C array
sizeof(a); // 5 * sizeof(int) = 20
```

Unlike raw arrays: can be returned from functions, passed by value, compared with ==, and does **not** decay to a pointer.

`std::array` — fixed-size contiguous storage (2/2)



insert: **Not supported** — size fixed at compile time

delete: **Not supported** — size fixed at compile time

search: Linear scan $O(n)$ — no built-in ordering

std::vector — dynamic contiguous storage

std::vector<T> is a resizable array:

```
#include <vector>

std::vector<int> v; // empty
v.push_back(10); // v: {10}
v.push_back(20); // v: {10, 20}
v.push_back(30); // v: {10, 20, 30}

v.size(); // 3
v.capacity(); // >= 3 (may be 4)

v[1]; // 20
v.at(1); // 20 (bounds-checked)
v.front(); // 10
v.back(); // 30

v.pop_back(); // v: {10, 20}
```

push_back is amortised $O(1)$. Random access is $O(1)$. Insertion in the middle is $O(n)$.

std::vector — construction

Several ways to create a vector:

```
std::vector<int> v1{2, 3, 4}; // 3 elements: [2, 3, 4]

std::vector<int> v2(5); // 5 zeros: [0, 0, 0, 0, 0]
std::vector<int> v3(5, -1); // 5 copies: [-1, -1, -1, -1, -1]

std::vector<int> v4{5}; // 1 element: [5] NOT 5 zeros!
std::vector<int> v5(v1); // copy of v1: [2, 3, 4]
```

Predict the output before you run it:

```
std::vector<int> mystery(3, 7);
mystery.push_back(1);
// What is mystery.size()? ???
// What is mystery[0]? ???
// What is mystery.back()? ???
```

std::vector — capacity vs. size

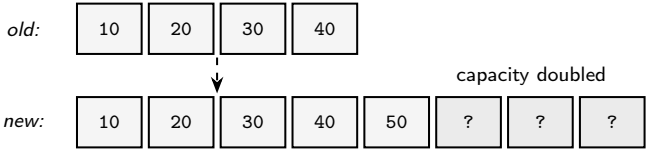
```
std::vector<int> v;
v.reserve(100); // allocate room for 100 ints
v.size(); // 0 --- no elements yet!
v.capacity(); // >= 100

for (int i = 0; i < 5; i++)
    v.push_back(i); // size grows: 1, 2, 3, 4, 5

v.size(); // 5
v.capacity(); // still >= 100

v.shrink_to_fit(); // request: reallocate to size == capacity
v.clear(); // size == 0, capacity unchanged
```

Key distinction: `size()` = how many elements are stored; `capacity()` = how many could fit before the next reallocation.

Why it matters: reallocation copies all elements ($O(n)$) and invalidates *every* iterator and pointer into the vector.

`std::vector` — growth strategy

When `push_back` exceeds capacity, the vector allocates a new, larger buffer and moves all elements:

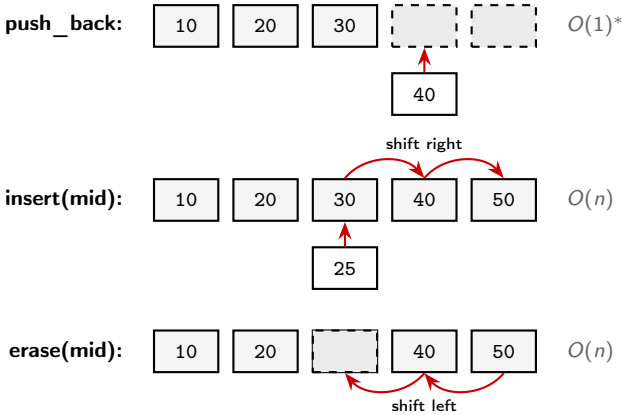


Typical growth factor: $\times 2$ (GCC/Clang) or $\times 1.5$ (MSVC).

Amortised $O(1)$: each element is moved at most $O(\log n)$ times across all `push_back` calls.

Best practice: if you know the final size, call `reserve()` up front to avoid all reallocations.

std::vector



access: `v[i]` jumps directly $\rightarrow O(1)$

search: linear scan $O(n)$, or $O(\log n)$ if sorted (`binary_search`)

std::deque — double-ended queue

`std::deque<T>` supports efficient insertion at *both* ends:

```
#include <deque>

std::deque<int> dq = {2, 3, 4};

dq.push_front(1); // dq: {1, 2, 3, 4}
dq.push_back(5); // dq: {1, 2, 3, 4, 5}
dq[0]; // 1 (random access OK)

dq.pop_front(); // dq: {2, 3, 4, 5}
dq.pop_back(); // dq: {2, 3, 4}
```

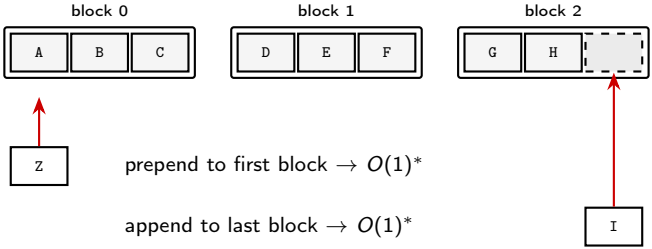
Internally: an array of pointers to fixed-size blocks.

Random access is $O(1)$ but with a higher constant than `vector`.

When to use: you need fast insertion/removal at *both* ends.

std::deque

layout:



push_front: prepend to first block $\rightarrow O(1)^*$

push_back: append to last block $\rightarrow O(1)^*$

access: `dq[5]`: block $= 5 \div 3 = 1$, offset $= 5 \bmod 3 = 2$ $\rightarrow$ `F` — $O(1)$

insert(mid): shifts elements within / across blocks $\rightarrow O(n)$

erase(mid): shifts elements to fill gap $\rightarrow O(n)$ (erase at ends: $O(1)$)

iterate: sequential through blocks, left to right $\rightarrow O(n)$

`std::list` — doubly-linked list

`std::list<T>` supports $O(1)$ insertion/removal *anywhere* (given an iterator):

```
#include <list>

std::list<int> lst = {2, 3, 2};

lst.push_back(5); // {2, 3, 2, 5}
lst.push_front(1); // {1, 2, 3, 2, 5}

lst.sort(); // {1, 2, 2, 3, 5} (its own sort!)
lst.unique(); // {1, 2, 3, 5}
lst.reverse(); // {5, 3, 2, 1}
lst.remove(2); // {5, 3, 1}
```

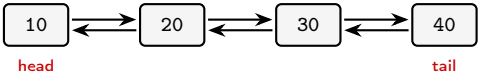
No `operator[]` — only sequential (bidirectional) access.

Cannot use `std::sort` — use the member `lst.sort()` instead.

When to use: frequent insertion/removal in the middle, and you already hold an iterator to the position. In practice, `vector` often wins due to cache effects.

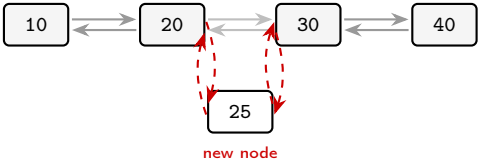
std::list

iterate:



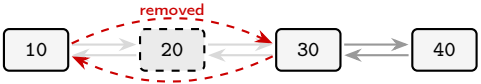
$O(n)$

insert:



$O(1)$ *rewire*

erase:



$O(1)$ *rewire*

access: No `operator[]`; walk from `begin()` $\rightarrow O(n)$

search: Linear scan only $\rightarrow O(n)$

std::forward_list — singly-linked list

Minimal-overhead singly-linked list (no `size()`, no `push_back`):

```
#include <forward_list>

std::forward_list<int> fl = {2, 3, 2};

fl.push_front(5); // {5, 2, 3, 2}
fl.insert_after(fl.begin(), 0); // {5, 0, 2, 3, 2}
fl.erase_after(fl.begin()); // {5, 2, 3, 2}
fl.remove(2); // {5, 3}
fl.sort(); // {3, 5}
fl.reverse(); // {5, 3}
```

Only forward traversal. All operations work relative to `before_begin()` or a specific position.

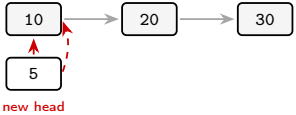
When to use: memory-constrained scenarios where you only need forward traversal (rare in practice).

std::forward_list

iterate:

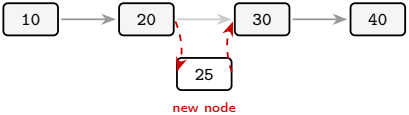
$O(n)$ *forward only*

push_front:



$O(1)$

insert_after:



$O(1)$ *rewire*

erase_after:

$O(1)$ *rewire*

access: No `operator[]`; walk from `begin()` $\rightarrow O(n)$

search: Forward scan only $\rightarrow O(n)$

Sequence container complexity

	`[]` / `at`	`push_back`	`push_front`	`insert(it)`	`erase(it)`
`array`	$O(1)$	—	—	—	—
`vector`	$O(1)$	$O(1)^*$	—	$O(n)$	$O(n)$
`deque`	$O(1)$	$O(1)^*$	$O(1)^*$	$O(n)$	$O(n)$
`list`	—	$O(1)$	$O(1)$	$O(1)$	$O(1)$
`forward_list`	—	—	$O(1)$	$O(1)$	$O(1)$

*Amortised.

For `list` / `forward_list`, insert/erase are $O(1)$ *given an iterator to the position*. Finding that position is $O(n)$ if you don't already have it.